SPVS final setup and demonstration guide

Current local QR/audio checkpoint

The 27 October runtime update has already backed up/migrated the local database, refreshed dependencies and built Android, iOS simulator and signed-device clients with SpvsAcoustic. The new apps were not installed on phones. Wait for the user's explicit classroom readiness before installation or physical trials; old installed clients cannot complete policy v2.

For the next attendance test, use a fresh session and start the faculty QR/audio controls. The student scans QR, requests microphone access, starts listening, and identifies their queued attempt; faculty plays that signal before face and location capture proceed. See the detailed physical plan. Do not enable iOS dynamic or required attestation policies merely to make a demo pass; the local review choices were retained. Database records and raw/private calibration inputs are local state, not part of Git synchronization.

See iOS Mac validation for native setup and iPhone observations, and Android physical validation for the completed hostel demo through manual review and phone history.

Use the **SPVS: dashboard** task for the local faculty UI at `http://127.0.0.1:5173`. The API and Metro tasks must also be running for phone testing. The Motorola's Wi-Fi must actually be connected to the Mac's reachable network; enabling Wi-Fi without joining it is insufficient. The dedicated Android demo student was created in the local database and is not automatically created by a fresh clone/seed; create a separate student through Administration and enroll it in the test course when preserving an existing iPhone registration.

Use the source ZIP from this completion branch and read FINAL_COMPLETION_REPORT.md before presenting capabilities. Keep the baseline release intact. Commands below are for a MacBook Air M4 with macOS and a trusted development network. The Linux execution report does not claim these Mac commands were run on Apple Silicon.

1. Prerequisites

Install Homebrew from its official instructions, Docker Desktop for Apple Silicon, Android Studio for Apple Silicon, and Xcode from the App Store. Launch each application once, accept its license and complete first-run components. In Xcode Settings > Locations select the active command-line tools. Then install:

```
brew install python@3.12 node@22 uv openjdk@17 ruby
export PATH="$(brew --prefix node@22)/bin:$(brew --prefix ruby)/bin:$PATH"
export JAVA_HOME="$(brew --prefix openjdk@17)/libexec/openjdk.jdk/Contents/Home"
export ANDROID_HOME="$HOME/Library/Android/sdk"
export PATH="$ANDROID_HOME/platform-tools:$PATH"
xcodebuild -version
node --version
java -version
```

Use Python 3.12, Node 22 (or the verified Node 24 environment), JDK 17 and the pinned npm/uv lockfiles. React Native's checked-in helper requires at least Xcode 16.1; install a current compatible Xcode and supported iOS simulator runtime. In Android Studio's SDK Manager install the SDK, build tools, CMake and NDK versions reported by the generated project. This run used Android API/build tools 37, Gradle 9.4.1 and NDK 27.1.12297006. Let official tools install missing exact versions. Do not substitute unofficial Maven mirrors or disable TLS.

2. Local services and accounts

From the unpacked `spvs-root` directory:

```
python3 scripts/init_env.py
uv sync --project backend --frozen
python3 scripts/download_models.py
npm ci --prefix dashboard
```

```
npm ci --prefix mobile
docker compose up -d db --wait
bash scripts/backend.sh alembic upgrade head
bash scripts/backend.sh alembic check
bash scripts/backend.sh python -m app.seed
```

The setup script creates local `.env` secrets and a random DEMO_PASSWORD. Read that password locally; never paste it into reports or commit the file. Demo accounts are admin@spvs.example, faculty@spvs.example and student@spvs.example. Seed creates no attendance or biometrics. Change the demo classroom to real coordinates and permitted real AP references before phone testing.

Run in separate terminals:

```
make api
make dashboard
```

The API listens on port 8000 and the dashboard on 5173. Alternatively use `docker compose up --build -d` after models are downloaded. Open http://localhost:5173. `/health` checks PostgreSQL and `/ready` also loads the real models. Use trusted TLS for any real data. Create individual accounts for institutional testing instead of shared demo credentials.

3. Automated verification

Create an isolated test database once, without changing the application database:

```
docker compose exec db createdb -U spvs spvs_test
bash scripts/backend.sh pytest
npm test --prefix dashboard -- --run
npm test --prefix mobile -- --run
npm run build --prefix dashboard
npm run typecheck --prefix mobile
cd mobile
npx expo export --platform android
npx expo export --platform ios
cd ../dashboard
npx playwright install chromium
npx playwright test
```

On macOS the configuration uses Playwright's installed Chromium. Set SPVS_CHROMIUM_PATH only when explicitly selecting another executable. The browser tests read the local demo password in memory, exercise real API endpoints and change only demo data. PostgreSQL tests require a database name ending `_test` and truncate its tables. Unit/contract tests use synthetic sensor and attestation inputs; they are not physical-device verification.

4. Android debug build and run

For emulator-only development set EXPO_PUBLIC_API_URL to http://10.0.2.2:8000/api/v1; for a physical phone set a LAN-reachable server URL. For real data use trusted HTTPS. Set SPVS_DEV_HTTP=1 only for a local HTTP development build.

```
export EXPO_PUBLIC_API_URL=http://10.0.2.2:8000/api/v1
export SPVS_DEV_HTTP=1
bash scripts/build_mobile_macos.sh android
cd mobile
npx expo start --dev-client
```

The script runs a genuine Gradle assembleDebug and copies its output to release/final/SPVS_ANDROID_DEBUG.apk only on success. Install with `adb install -r android/app/build/outputs/apk/debug/app-debug.apk` from mobile, or use Android Studio. The debug development client needs Metro; it is not a signed production app. Use `adb reverse tcp:8081 tcp:8081` when developing over USB. Configure the API endpoint consistently in the terminal running Metro.

The 24 October Mac Android build also works with official command-line SDK tools; Android Studio is not required for that path. The build script discovers Homebrew Java 17 and `.runtime/android-sdk` when available, while respecting explicit

`JAVA_HOME` and `ANDROID_HOME` settings. The saved **SPVS: Android debug build** task targets arm64-v8a. Use `SPVS_ANDROID_ARCHITECTURES` to select another supported architecture. After installation, sign in and use **Settings** → **Wi-Fi setup check** before attempting attendance. Register actual classroom AP references using measured data.

For release distribution remove SPVS_DEV_HTTP, set an HTTPS API URL, configure institution-owned signing keys outside Git and build a reviewed release variant. Never distribute the debug signing key as an institutional credential.

5. iOS project, CocoaPods and simulator

```
cd mobile
brew install ruby@3.3
export PATH="/opt/homebrew/opt/ruby@3.3/bin:$PATH"
export BUNDLE_PATH="$PWD/../.runtime/bundle"
bundle install
cd ..
export EXPO_PUBLIC_API_URL=http://localhost:8000/api/v1
export SPVS_DEV_HTTP=1
bash scripts/build_mobile_macos.sh ios-simulator
cd mobile
npx expo run:ios
```

The script generates `ios/SPVSStudent.xcworkspace` with `--no-clean`, installs pods and invokes xcodebuild with simulator SDK and CODE_SIGNING_ALLOWED=NO. The simulator can test navigation/network/permission states; it cannot establish Secure Enclave, App Attest, live-camera liveness or real Wi-Fi behavior. The resolved Ruby and CocoaPods lockfiles are now retained from the Mac run; earlier Linux results did not establish iOS compilation.

6. Physical iPhone and App Attest

Set an HTTPS API endpoint reachable from the phone, then:

```
unset SPVS_DEV_HTTP
export EXPO_PUBLIC_API_URL=https://YOUR-INSTITUTION-HOST/api/v1
export SPVS_APP_ATTEST_ENV=development
bash scripts/build_mobile_macos.sh ios-device
```

In Xcode select the SPVSStudent target, set a development team, verify bundle ID/capabilities and select a connected iPhone. Enable Developer Mode on the phone and trust the developer profile if requested. Press Run. Start Metro with the same API environment. Camera and location require user consent; select Precise Location. Configure App Attest server trust settings and Apple Developer capabilities exactly as described in CROSS_PLATFORM_IMPLEMENTATION.md. Unsupported or unconfigured App Attest remains unverified. Handle a key-lost error through administrator device reset; never silently replace the registered key.

EAS is optional. The checked-in eas.json defines development, simulator, preview and production profiles. Run `eas init` in your own account, configure EXPO_PUBLIC_API_URL for the selected profile and run `eas build --platform ios --profile development`. Keep signing credentials in EAS or the Keychain. For TestFlight create an App Store Connect app, use production App Attest entitlement/server environment, increment build version, produce a genuine signed archive with Xcode Organizer or EAS production, upload and complete Apple's review/export-compliance steps. No IPA is included unless actually compiled and signed.

7. Demonstration sequence

1. Sign in as administrator, configure a real room, enrollment and the separate Android/iPhone presence policies. Keep iPhone review as the safe default.

2. Sign in as faculty and create a currently active session.

3. On a physical phone sign in as student and register a native device key with password confirmation. Inspect the reported capability in the admin dashboard.

4. Consent to biometric enrollment, permit the camera and follow the randomized center/head-turn/blink prompts. Show the actual result, including retry failures.

5. Select the session and complete fresh identity and location capture. Demonstrate that iPhone current-network data is not a Wi-Fi fingerprint and that review is not attendance approval.

6. Inspect faculty attempts, attendance records, audit events and CSV exports. Test invalid signature/replay using automated tests; do not fake a physical success.

7. Demonstrate face deletion/re-enrollment and an audited lost-device reset. Show pending real-device/provider/calibration work honestly.

8. Troubleshooting

- API unreachable on phone: localhost means the phone itself. Use the Mac LAN IP or trusted HTTPS host, bind API to 0.0.0.0, check firewall and same-network routing. Development HTTP requires a freshly generated development configuration; production remains HTTPS-only.

- Expo Go or missing native module: install a generated development client. Restarting Metro cannot add native modules to an existing binary.

- iPhone Wi-Fi unavailable: verify entitlement, provisioning and Apple's location/privacy requirements. Do not insert dummy BSSID/RSSI values; use review or retry.

- Reduced GPS/poor accuracy: enable Precise Location and retry with a fresh fix. Indoor GPS may remain insufficient.

- Liveness failures: use even light, one face, slow blink and steady capture. Inspect actual timing/orientation on each device. Do not lower mandatory identity rules to force a demo.

- Gradle 403/network/TLS failures: inspect the exact official repository URL. Allow it through the institutional network and configure the JVM proxy/CA correctly. Preserve TLS and artifact verification. Do not mirror an unverified AAR manually.

- Xcode/CocoaPods errors: verify active Xcode, compatible simulator runtime, Ruby/Bundler and generated podspecs. Source code or prebuild success does not prove native compilation.

- App Attest rejected: check TeamID.bundleID, environment, official root fingerprint, matching entitlement and physical-device support. A verified synthetic test certificate is not an Apple attestation.

Policy v2 retires the iOS gps_only option. Current iOS choices are review, retry and dynamic; the migration maps an existing gps_only choice to dynamic while retaining stricter review/retry settings. Dynamic requires completed QR/audio proof alongside mandatory identity/device/GPS gates; a global requirement for attestation still applies. The local review policy was not changed. QR/audio does not independently attest a platform or sensor and can be relayed. See the protocol.

Current SPVS status

Checkpoint: **27 October 2026, Asia/Kolkata**. Publication: PR #7, building on the upstream QR/acoustic PR #6. The PR/Git refs are authoritative for current publication state.

Read runtime integration, C003 observations, current verification, and synchronization.

Area	Current state
Integration checkpoint	`0fc235f916fb662db71c298091255ec10f0917f6`, based on upstream main `404eed8`
New attendance policy	Mandatory rotating QR + acoustic proof, alongside existing identity/device/GPS/platform-presence gates
Local database	Backed up privately; migrated to `9bd32a88f101`; existing counts and records preserved; no schema drift
Services	API/dashboard restarted; health/readiness and new presence route verified
Local automated tests	146 backend + 64 mobile + 11 dashboard + 10 native Wi-Fi = 231 passed
Separate upstream evidence	3 browser tests passed in cloud; not rerun against the live local demo database
Native builds	New Android arm64 APK, iOS simulator and signed-device app built with SpvsAcoustic; signatures/configuration checked
Device installation / new physical tests	Not performed; wait for explicit user readiness in the classroom
C003 configuration	Separate classroom, 50 m radius, two provisional 5 GHz references at -70 dBm; one of two matches suffices under existing rule
Earlier C003 inside trial	Automatic PRESENT / VERIFICATION; Wi-Fi ratio 0.5; pre-v2 flow
Later distance trial	User-confirmed hostel attempt rejected OUTSIDE_GEOFENCE at reported 151.1 m; Wi-Fi was not evaluated
Near-door validation	Calibration scan collected with door closed; fresh Wi-Fi-specific attendance comparison not completed
Earlier Android demo	Registration/enrollment, pending submission, manual CORRECTION, resolved dispute and phone history observed
iOS policy	Local review retained; dynamic requires a separate audited choice; App Attest remains unconfigured
Fresh npm audit	4 high mobile affected entries, zero dashboard; backend vulnerability scan historical
Production	Real QR/audio interoperability, leakage/relay, calibration and operational release gates remain open

Earlier dated reports describe their original checkpoints. Neither the old automatic C003 result nor native compilation proves physical QR/audio success. The old installed apps must be updated before the next trial. No physical test, session challenge or audible playback was started during this synchronization.

Printable records: runtime PDF, C003 PDF, sync PDF. See test results, physical plan, and release inventory.

Git publication excludes private backup/database data, credentials, signing keys, raw network identifiers, exact coordinates, templates, frames/audio and raw UI captures. It does not deploy a live cloud service or replicate the local database.

Deployment guide

Local prerequisites and compatibility

Use Python 3.12, `uv`, Node 24 LTS (or a compatible Node 22 version meeting the locked tool engines), Docker with Compose, and Git. On Apple Silicon, install native ARM64 Python/Node and Docker Desktop for Apple Silicon. PostgreSQL 16 has ARM64 images. OpenCV and MediaPipe wheels must match the interpreter and architecture; avoid mixing x86 Homebrew libraries with ARM64 wheels. This environment's measured platform is Linux x86_64, so macOS native validation remains a manual task.

The normal setup is `make setup`, `make db`, `make migrate`, `make seed`, followed by `make api` and `make dashboard`. The generated `.env` contains unique local secrets. Protect it with restrictive permissions and never include it in an academic submission. For phones, configure `mobile/.env` with a reachable API URL; it contains no credentials.

Linux hosts also need `libgl1`, `libgles2`, `libegl1` and `libglib2.0-0`. The Dockerfile installs these through the distribution package manager. In the restricted cloud workspace, `scripts/cloud_install.sh` extracts verified distribution libraries from a temporary container into ignored local runtime storage; `scripts/backend.sh` activates that location. It does not replace system libraries or disable package/TLS verification. On a normal Debian/Ubuntu machine, installing the packages with the system package manager is simpler.

Docker workflow

1. Generate `.env` and checksum-verified face models.

2. Run `docker compose up --build -d`.

3. Run `docker compose exec backend python -m app.seed` for local demonstration accounts only.

4. Check API `/health` and `/ready`; readiness must load all model assets.

5. Sign into the dashboard and configure real classroom data.

The backend image uses a pinned uv executable and a frozen Python lockfile. The web image uses `npm ci` and nginx. PostgreSQL has a health check and a persistent named volume. API migrations run before uvicorn at container startup. Persistent database files and model files survive process restarts; HTTP processes and authentication sessions in browser memory do not.

Corporate/cloud TLS interception may require the existing trusted CA bundle inside build steps. Both Dockerfiles support an optional BuildKit secret named `ca_bundle`. The cloud installer supplies the host's trusted public CA bundle using `--secret`; it is not baked into the image or included in artifacts. Normal direct-Internet Compose builds need no extra secret. Never use insecure TLS flags as a workaround.

Production-inspired deployment changes

The Compose configuration is a development baseline. Add trusted HTTPS termination, a non-demo administrator provisioning process, institution-specific CORS origins, encrypted backups, key rotation and a retention policy. Remove demo accounts or replace their passwords before real use. Keep the database port private. Use a separate least-privilege application database role and reviewed migration credentials; the development `spvs` role is also the database owner.

Verify `JWT_SECRET` and `BIOMETRIC_KEY` are independent, generated secrets delivered through a secret manager. Rotating the biometric key without re-encryption makes stored templates/device keys unreadable. Do not invent credentials for provider attestation. Add a provider-backed verifier only with real Google/Apple project credentials and package/signing configuration.

Android development build

Install Android Studio and its supported SDK/NDK tools, enable USB debugging on a test phone and run `npm run android` in `mobile`. The generated project declares the exact SDK/Gradle versions selected by the locked Expo release. `expo prebuild --platform android --no-install` regenerates native code and discovers the local Wi-Fi module. Do not use Expo Go for that module. Gradle compilation, a Metro/Hermes export and physical device tests are distinct checks.

Use a trusted HTTPS URL for distribution. Development HTTP on a local network is only for controlled local testing. Do not put a raw API secret or biometric key in `EXPO_PUBLIC_*`: those values are embedded in the client bundle. iOS builds require Xcode/signing on macOS, and the application reports Wi-Fi scanning as unsupported.

Troubleshooting

Symptom	Diagnosis / action
Database connection failure	Start Compose database, confirm port availability and `.env` URL
Backend config validation fails	Generate real JWT/Fernet secrets; placeholders are intentionally invalid
Readiness returns 503	Run verified model downloader; inspect missing assets and writable caches
Official downloads denied	Allow `media.githubusercontent.com` and `storage.googleapis.com` in environment settings
Model checksum mismatch	Stop using the file; retry official source and investigate upstream provenance
Phone cannot reach API	Use LAN address/HTTPS; `localhost` on a phone points to the phone
Wi-Fi unavailable	Confirm native build, location permission/services and Android scan throttling; follow policy
Frequent liveness retries	Evaluate frame timing, slow blink, pose direction and lighting on the actual device
Browser session disappears on reload	Intended in-memory token storage; sign in again
Duplicate request conflicts	Preserve exact signed payload/key for a retry; use new challenge for new evidence

Do not delete the PostgreSQL volume to resolve ordinary setup issues. `docker compose down -v` is destructive and is not part of normal setup or restart instructions.

Android dependency download in this cloud

The observed Gradle build used Java 17, Gradle 9.4.1, Android API/build tools 37 and NDK 27.1.12297006. React Native 0.88.0's AAR redirects from Maven Central to `repo.reactnative.dev`; allow that specific domain if the proxy returns HTTP 403. The saved cloud configuration includes this addition, but saving a draft does not apply it. Review/save the environment settings and publish before retrying. Keep TLS verification enabled, using the environment's trusted Java CA store where necessary. Full native compilation and device execution remain unverified in the delivered results.

Cross-platform release configuration

Apply migrations through `7a2d13ce4f90` before deploying updated clients. Leave ALLOW_LEGACY_DEVICE_REGISTRATION=false; migrate existing HMAC devices through approved resets. Configure ios_presence explicitly, retaining review by default. Set APPLE_APP_ID, APPLE_ATTEST_ENVIRONMENT, APPLE_ATTEST_ROOT_PEM and APPLE_ATTEST_ROOT_SHA256 only after verifying Apple Developer configuration and the official trust anchor; mount the PEM read-only into the backend container. A successful /ready response proves model/database availability, not platform attestation deployment.

Use HTTPS, institution-owned signing credentials and tested backups. Raw frames are transient; encrypted biometric templates persist until explicit deletion. Define and implement institution-approved retention for templates, coarse location summaries, security events and expired challenges before real deployment. Automated retention/cryptographic key rotation and load testing remain operational work; the academic package does not invent a compliance guarantee. Run the physical Android/iPhone matrix and genuine App Attest interoperability checks before enabling production policy. EAS/TestFlight instructions are in SETUP_AND_DEMO_GUIDE.md.

User manual

Administrator

Sign in using the dashboard. In Administration, create staff and student accounts with unique emails and student roll numbers. Assign the faculty account when creating a course, then enroll the correct students. The current role is enforced immediately by the backend; changing a role revokes refresh sessions and device binding and requires an explanation.

In Classrooms, enter surveyed coordinates and a suitable radius. A radius must be 5–300 m. Enter actual Wi-Fi BSSIDs and minimum RSSI values only after collecting a campus reference profile; each line is `aa:bb:cc:dd:ee:ff,-80`. Leave the list empty when unknown, then select a deliberate unavailable policy. The default is manual review. Do not label the demo coordinates as real campus measurements.

Device reset requests appear in Administration. Verify student identity before approving. An approval revokes the previous device; the student must then register the replacement phone. Policy changes are recorded with the old and new values. Requiring attestation while its provider is unconfigured results in review, not silent bypass.

Faculty

Student **Request a review** creates a dispute; it does not approve attendance. After reviewing the evidence and deciding a change is justified, use **Corrections & reviews** → **New correction**, select the correct session/student, choose the justified status and enter a reason. Then use the dispute's **Resolve** action to record its resolution. These are separate actions in the current UI. The official record has source CORRECTION; the original automated attempt and its reasons remain unchanged. Resolving a dispute alone does not create attendance.

Courses lists assigned courses and enrolled students. Sessions opens a bounded attendance window for one course and classroom. Choose local start/end times; the API receives timezone-aware ISO timestamps. Session details refresh approximately every 15 seconds and show attempts separately from official records. Close a session to stop further automatic attendance.

Inspect reasons and risk indicators before reviewing an attempt. In Corrections & reviews, choose the session and student, enter PRESENT/ABSENT/EXCUSED and a meaningful reason. A correction is visibly sourced as CORRECTION and retains its previous status. Resolving a dispute records an explanation; it does not itself create attendance. Make a separate correction if justified. Reports downloads actual authorized records as CSV.

Student

Install the development app and sign in. First register the phone using account-password confirmation. Then open Face enrollment, read the retention explanation and grant consent. Follow the center, turn and slow-blink prompts in even lighting. The server rejects zero/multiple faces, weak samples or incomplete action sequences.

Enter the institution email without embedded spaces. The sign-in form uses an email keyboard and reports malformed email/password fields explicitly. A format error is distinct from an incorrect password. Never enter the `DEMO_PASSWORD=` label itself when using a local demo password; enter only its value.

After faculty correction, refresh **My history**. Check **Official records** for the current status and source. The original attempt may still show PENDING REVIEW below it; this preserves the automated decision and does not erase the correction.

For an open session, faculty must start the classroom challenges on the session page. Tap Verify my attendance, scan the current QR, allow foreground microphone access, and tap Listen. Read the displayed attempt label to faculty so they play your queued signal. Keep the app open and disconnect Bluetooth headphones. Only server-verified sound advances to face/liveness capture, location and Wi-Fi. PCM is processed transiently on the phone, not uploaded or saved. If QR/audio is unavailable or inaccessible, use Finish attempt & request faculty review; that does not automatically approve

attendance. Retry an identical signed response when offered after a network failure. See the physical guide.

Faculty plays one queued attempt at a time. Stopping/restarting classroom challenges invalidates earlier proofs. The new flow requires a rebuilt native client with SpvsAcoustic; an old installed APK cannot gain microphone support from a JavaScript reload alone.

History shows official records and attempts separately. Request review of a disputed attempt with an explanation. A PENDING_REVIEW result is not confirmed presence. Settings opens OS permission controls and signs out. Enrollment allows password-confirmed template deletion; Device allows a replacement request. No background location collection is implemented.

Settings → **Wi-Fi setup check** displays available access-point IDs and measured signal strengths on Android, or permitted connected-network information on iPhone. It does not upload observations, require device registration, or record attendance. Confirm classroom AP identities with an administrator before using them as references. Enable precise location and location services; repeated scans may be throttled. An unavailable result is not proof that Wi-Fi is switched off.

Result interpretation

State	Meaning	Next action
PRESENT	All configured gates passed and attendance was recorded	Check history
REJECTED	A mandatory condition failed or a duplicate record exists	Inspect reason; request review if disputed
PENDING_REVIEW	Location/signals/attestation are uncertain under policy	Contact faculty; do not assume attendance approval
REQUIRES_RETRY	Fix a recoverable evidence issue	Enable permissions or obtain a better GPS fix, then retry
EXPIRED	Session/challenge is no longer valid	Ask faculty about a new session or review

Cross-platform completion update

On iPhone allow Camera and Precise Location. SPVS may read the current Wi-Fi BSSID when Apple permits it; it cannot scan nearby networks. Attendance can legitimately remain PENDING_REVIEW under the institution's iPhone policy. Administrators configure iPhone presence independently from Android missing-Wi-Fi policy, with an audit reason. The Device capabilities panel distinguishes legacy signing, reported native storage and verified App Attest registration. Register Apple app attestation only after institutional setup. Reinstall/key loss requires the existing reset-review workflow. To re-enroll a face, delete the existing template with password confirmation, consent again, then capture a new live sequence.

Policy v2 retires the iOS gps_only option. Current iOS choices are review, retry and dynamic; the migration maps an existing gps_only choice to dynamic while retaining stricter review/retry settings. Dynamic requires completed QR/audio proof alongside mandatory identity/device/GPS gates; a global requirement for attestation still applies. The local review policy was not changed. QR/audio does not independently attest a platform or sensor and can be relayed. See the protocol.

Demonstration script

Current policy-v2 demonstration order

Physical QR/audio testing has not started. Resume only when the user confirms classroom readiness, with rebuilt native apps installed and a fresh session. Faculty starts challenges, displays the rotating QR, and plays one matching queued attempt after the student taps Listen. Camera/microphone permissions and audible accessibility/review alternatives must be handled. Successful QR/audio then leads to face/liveness/GPS/Wi-Fi; none of those gates are waived.

The earlier C003 inside automatic pass and hostel GPS rejection are historical pre-v2 observations. Do not present them as new-flow validation. Record which gate determines outside outcomes, including QR/audio leakage and real-time relay risks. See C003 evidence and the protocol test plan. The manual-review steps below still apply after a justified review decision.

Current observed hardware flow is recorded in Android device validation and iOS validation. Use current status for the latest evidence rather than the historical baseline reports.

Reproduce the manual-review demonstration

1. Use a dedicated test student so an existing registered phone remains intact.

2. Complete native registration, consented enrollment and a real session submission.

3. Show the actual pending decision and evidence; missing classroom references

can produce WIFI_UNAVAILABLE even when the scan itself is available.

4. Request review from the phone. Show that a dispute exists but no official

attendance record is created by that request.

5. As assigned faculty, review the evidence. For an explicitly controlled demo,

label any correction reason as a demo and state what was not verified.

6. Create the justified attendance correction, then separately resolve the dispute.

The latest observed test exercised these authenticated faculty API operations; a complete faculty-browser click-through remains to be tested.

7. Refresh student history. Show PRESENT with source CORRECTION and the original

PENDING REVIEW attempt. Do not describe this as automatic classroom verification.

The Motorola must be connected to the Mac's reachable Wi-Fi network for the local API. The latest history check recovered after Wi-Fi reconnection; no fake data or policy relaxation was used. The hostel scan test is not actual classroom calibration.

Before the evaluation

Run the automated suites and retain their current result files. Configure a real classroom and a deliberate Wi-Fi fallback. Complete the real-device checklist with consenting participants before promising a successful live capture. Prepare the randomly generated demo password locally; never place it in slides or the ZIP.

Eight-minute desktop demonstration

1. Explain the two-layer design and the reason uncertain signals produce review.

2. Sign in as administrator. Show the actual course, enrolled demo student and classroom configuration. Point out that initial metrics are real counts rather than seeded attendance.

3. Open a faculty session for the assigned course and a short future window.

4. Show the session page, separate attempts and official record tables, and automatic refresh.

5. Show the policy's manual-review Wi-Fi default and unconfigured attestation status.

6. Sign in as faculty and demonstrate that administrator navigation and API permissions are unavailable. An automated test also verifies cross-course denial.

7. Explain an authorized correction, enter a reason only for a legitimately reviewed record, and show its source/history. Do not misrepresent the correction as biometric verification.

8. Close the session and export the actual CSV. Open the security event log and identify session/correction/export events.

Phone demonstration after hardware validation

Sign in as a student, register the phone, consent to face enrollment and follow the randomized prompts. Open the active session, capture a fresh sequence, collect real location and available Wi-Fi, and show the exact server result. A review/retry result is a valid demonstration of the decision policy; never switch to fake evidence to force PRESENT.

Demonstrate one rejected or reviewable condition: permission denial, a clearly outside location, a revoked device, or a static-photo liveness failure. Then show the corresponding attempt, reason and dispute workflow. Any successful person-to-session result must come from actual captured evidence or be visibly identified as a faculty correction.

If hardware or network is unavailable

Demonstrate the running web/API/database workflow, real-model smoke tests and signed-request protocol tests. State that the API happy-path tests isolate vision and that physical sensor/liveness performance has not been measured. Show `SPVS_VERIFICATION_REPORT.md`, not an invented success video.

Cross-platform completion update

For the cross-platform extension, show Android and iPhone capability differences before a phone demonstration. Open Administration and show the distinct iPhone review/retry/GPS-only choices and required audit reason. Show native device key algorithm and label storage as reported. If no Apple-configured physical device is available, show the explicit unconfigured/unsupported state and synthetic test evidence; do not claim an App Attest demo. Only mark real enrollment and attendance complete after a physical camera run succeeds. Present the final test matrix and outstanding hardware gates alongside the APK, if built.

Current and historical test/build results

Current local checkpoint - 27 October 2026

Suite	Passed	Failures/errors	Skipped	Evidence
Backend	146	0	0	JUnit
Mobile	64	0	0	JUnit
Dashboard	11	0	0	JUnit
Native Wi-Fi	10	0	0	JUnit

231 local tests passed. TypeScript, dashboard build, both JS exports, migration/model drift, generated native configuration, Android arm64 APK/signature, iOS simulator and signed-device builds passed. SpvsAcoustic is present in the new APK and the iOS native compilation. No rebuilt phone installation or physical QR/audio test was performed. See local results.

The upstream cloud ran three browser tests and reports 234 total. Those browser and container checks were not rerun against the local demo database, and are separate from the local total. Fresh npm audits report four high mobile affected entries and zero dashboard findings; backend vulnerability scanning is historical.

Earlier C003 automatic inside attendance and the later hostel GPS rejection precede policy v2. See C003 validation. All further physical testing waits for explicit user readiness in the classroom.

Prior local checkpoint - 26 October 2026

Suite	Passed	Failures/errors	Skipped	Evidence
Full backend	120	0	0	JUnit
Mobile	46	0	0	JUnit
Dashboard	8	0	0	JUnit
Native Wi-Fi	10	0	0	JUnit

184 passed, all rerun for this checkpoint. Mobile TypeScript and dashboard production build passed. The APK/native build remains the verified 24 October development host; no native code changed, and current JavaScript is served by Metro. The physical Android demo reached manual faculty API correction, resolved dispute, attributable audit and verified phone history. Automatic classroom approval and faculty-browser click-through remain unverified. See physical evidence.

Fresh npm audits: mobile has 5 high affected package entries, dashboard has zero. Backend security auditing remains historical. Prior browser/container checks and native build runs retain their original dates; repeated tests are not added to this total. See current verification.

Prior local checkpoint - 24 October 2026

Suite	Passed	Failures/errors	Skipped	Evidence
Backend, full suite	120	0	0	JUnit
Mobile	42	0	0	JUnit
Dashboard	8	0	0	JUnit
Android native Wi-Fi	10	0	0	JUnit

Total: **180 passed** across these four suites. The backend/mobile/dashboard results were rerun for the audit; the native results are from the preceding verified local Android build. Historical browser tests are excluded from this total. Mobile TypeScript and the dashboard production build passed. Native Android compilation and APK signature verification passed; physical Android testing is pending. See Android validation.

iOS simulator and signed-device builds passed in the earlier local Mac run. Physical iPhone enrollment succeeded on 24 October; an attendance submission passed face/liveness/GPS but required review for iOS presence policy. No official attendance record was created. See iOS validation.

Fresh npm audits: mobile reports 5 high affected package entries; dashboard reports zero. Backend vulnerability auditing was not rerun locally. Upstream deprecation warnings and genuine provider-attestation/calibration gaps remain.

See the full audit for scope and provenance.

Historical cloud completion run - 23 October 2026

The tables and statements in this section describe that earlier run only.

Suite	Passed	Failures/errors	Skipped
Backend	109	0	0
Mobile	28	0	0
Dashboard	8	0	0
Browser	3	0	0

Total: **148 passed**. See FINAL_COMPLETION_REPORT.md for test boundaries, commands and intermediate failures. Baseline evidence is retained separately.

Capability	Observed status
Android debug APK	PASS: genuine Gradle assembleDebug, arm64-v8a; debug signed, development client; physical-device testing NOT RUN
Android/iOS JS export	PASS: separate platform exports; final combined export in evidence
iOS native project	PASS: Expo prebuild, camera/location/privacy settings, local podspec/autolinking and static configuration checks

Capability	Observed status
iOS native compilation / simulator / IPA	NOT RUN: Linux host; no macOS/Xcode or Apple signing identity
PostgreSQL migrations	PASS: fresh upgrade, check, full downgrade, re-upgrade, check on isolated PostgreSQL 16 database
Backend and dashboard containers	PASS: both images built and used for real browser/API tests
Physical Android / physical iPhone	NOT TESTED
Apple App Attest	Synthetic verifier/counter tests passed; genuine provider/device verification UNVERIFIED
Google Play Integrity	NOT CONFIGURED; fail-closed boundary only
Biometric calibration	NOT PERFORMED; six real-model smoke cases are not accuracy metrics

No physical Android/iPhone, iOS simulator, Xcode or genuine provider-attestation result is claimed. Synthetic API/sensor/certificate inputs and mocked React Native components are identified in their test modules.

Physical-device validation: QR + audible acoustic attendance

Status: not performed for this feature. Earlier SPVS hardware reports concern the older flow. Automated PCM tests, JS exports and Android compilation do not establish that a laptop-to-phone acoustic channel works in a real classroom.

The 27 October local integration has now applied the migration and rebuilt Android, iOS simulator and signed-device apps. Those builds were not installed on phones. Start physical tests only after the user confirms classroom readiness; preserve existing credentials with an in-place development-client update rather than uninstalling a registered app.

Setup on MacBook Air M4

Use a native ARM64 terminal, current Xcode/command-line tools, Docker Desktop, Node 24 LTS, Python 3.12 + uv, Android Studio/SDK, JDK 17, and the repository's Bundler/CocoaPods setup. Preserve your existing uncommitted work before switching. From your existing repository checkout:

```
git fetch origin
git switch main
git pull --ff-only
make setup
make db
make migrate
make seed
make build
```

The commands above cover a fresh setup. In the already configured local Mac environment, the database migration is complete; do not reinitialize its database or reseed merely to resume testing. Use the existing API/dashboard/Metro tasks and the verified rebuilt clients. Preserve the C003 references and prior records.

Keep development services in separate terminals:

```
make api
make dashboard
```

Run automated checks (the test database name must end `_test`):

```
docker compose exec -T db createdb -U spvs spvs_test  # once; skip if it exists
make test
(cd mobile && npm run export)
(cd dashboard && npx playwright install chromium && npx playwright test)
```

For a LAN HTTP development demo only, replace the address below with your Mac's reachable Wi-Fi address. The phone and Mac must reach each other; campus client isolation can block this. Use trusted HTTPS for release builds.

```
export EXPO_PUBLIC_API_URL="http://$(ipconfig getifaddr en0):8000/api/v1"
export SPVS_DEV_HTTP=1
bash scripts/build_mobile_macos.sh ios-simulator
bash scripts/build_mobile_macos.sh ios-device
# Xcode opens: choose your Apple team and connected iPhone 16 Pro, then Run.
# Separate terminal with the same environment:
(cd mobile && npm start)
```

For Motorola Edge 50 Neo, enable USB debugging and authorize the Mac:

```
export EXPO_PUBLIC_API_URL="http://$(ipconfig getifaddr en0):8000/api/v1"
export SPVS_DEV_HTTP=1
export SPVS_ANDROID_ARCHITECTURES=arm64-v8a
bash scripts/build_mobile_macos.sh android
adb devices -l
adb install -r mobile/android/app/build/outputs/apk/debug/app-debug.apk
```

```
(cd mobile && npm start)
```

These commands must create **new** native builds containing `SpvsAcoustic`. The historical release APK does not include this module. Do not commit local `.env`, private keys, profiles, databases, recordings or biometric material. If provisioning changes, follow the existing iOS guide. Connected-Wi-Fi entitlement and App Attest remain separate opt-ins; nearby Wi-Fi scanning is never available on iOS.

Baseline run

1. Use consented test accounts, one per device. Confirm enrollment and native key

registration; configure the actual classroom geofence. Keep mandatory checks on. Set iOS policy to `dynamic` through the audited admin policy editor. Keep Android Wi-Fi requirements explicit; collect real references if Wi-Fi is required.

2. Open the faculty session page. Start challenges; verify a new QR every five

seconds and automatic hiding at expiry or loss of connection. Confirm browser speakers work at comfortable volume before students start.

3. Select session on the phone, scan QR, tap Listen, and read the eight-character

attempt label to faculty. Play that matching queue entry. Stay foreground; disconnect headphones/Bluetooth routing. Only then complete face/liveness/GPS.

4. Confirm server PRESENT only after every mandatory gate passes. Confirm one

official record and one final attempt even when retrying a lost final response.

5. Deny microphone/camera, then use Finish attempt & request faculty review.

Confirm PENDING_REVIEW, identity NOT_COMPLETED, and no automatic official record. Exercise dispute, reasoned faculty correction and resolution; retain original attempt and correction provenance. Do not call a manual correction an automatic pass.

Measurement matrix

Run every row on **iPhone 16 Pro** and **Motorola Edge 50 Neo**. Record OS/build, phone orientation/case, laptop/speaker model, browser, volume, distance, ambient condition, network RTT, attempt ID, decoder outcome, server reason, and latency. Collect at least 20 trials per ordinary seating/ambient combination; retain failures. Use anonymous location labels instead of exact personal coordinates or identifiers.

Condition	Procedure and expected interpretation
Front/middle/back, corners, obstructed seats	Measure decode success and latency; failed capture never auto-passes
Quiet and loud classroom	Speech, fans and ordinary classroom noise; vary safe volume, never discomfort
Doorway, corridor, adjacent room (door open/closed)	Measure sound leakage; successful decode outside is possible and is a limitation
Poor QR lighting, glare, distance	Exercise torch, rescan, rotation overlap and expired screenshot rejection
Recorded audio	Old recording must fail in a fresh attempt; current recording replay within deadline may succeed and must be reported

Condition	Procedure and expected interpretation
Live QR sharing	Share a current screenshot; QR alone must never suffice; sharing within ten seconds may redeem QR
Real-time QR + audio relay	Consenting tester relays both; record success/latency without claiming relay resistance
Simultaneous listeners	Queue different attempt tags; each phone ignores the other's frame; two faculty tabs cannot overlap issuance
Camera/mic denied or revoked	Clear guidance, no automatic pass, bounded retry/review; restore permissions in Settings
Call, lock screen, background, Bluetooth route	Capture stops or fails; no fabricated success, no lingering recording indicator
Network disabled/delayed/dropped acknowledgement	Exact retries recover accepted evidence; late answers fail; final submission is idempotent
Session closed or display stopped/restarted	Pending and prior verified proofs cannot produce new automatic attendance
GPS inaccurate/outside/mock, wrong face, failed liveness, revoked device	Valid QR/audio cannot override the failing mandatory gate
VoiceOver/TalkBack and audio inaccessible	Controls are identifiable; obtain faculty review without requiring inaccessible audio
Restart app midway / exhaust attempt budget	No automatic attendance; bounded attempts and review remain enforceable

Do not upload recordings by default. If an approved acoustic study genuinely needs recordings, obtain specific consent, use separate secure storage and a deletion schedule; the application itself deliberately has no raw-audio upload route.

Release gate and result sheet

Define institutional thresholds **before** collecting results (in-room completion rate, latency distribution, acceptable leakage/relay risk, review burden). Do not select thresholds just to fit observed results. Report counts, median/p95 latency, all reason codes, failures and network conditions separately for both phones.

For each trial: `device | OS | build SHA | scenario | seat label | attempt ID | QR result | audio result | end-to-end latency | final state | reason | review source`. Mark native Xcode compilation, installation, permission behavior, real audio interoperability and end-to-end hardware attendance separately. Any unperformed item stays **NOT TESTED**. Physical leakage or real-time relay success must remain in the final limitations report, even when every automated test passes.